1. Beadandó feladat dokumentáció

Készítette:
Szép Tamás
E-mail: ko8dqj@inf.elte.hu

Feladat:

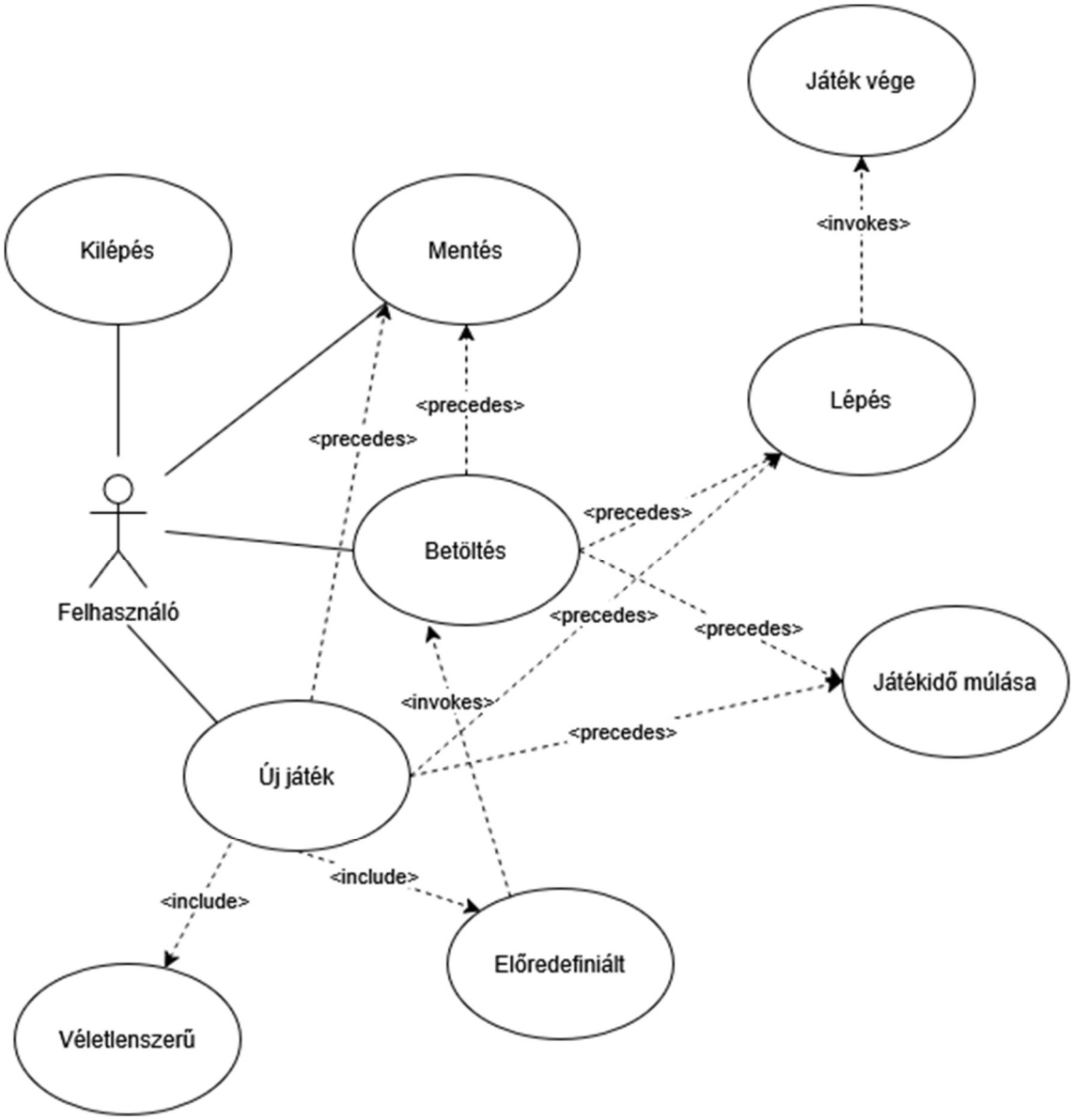
Készítsünk programot, amellyel a nonogram (más néven Hanjie vagy Griddlers) japán képrejtvényt oldhatjuk meg. Adott egy n × n mezőből álló tábla, amely egy rejtett fekete-fehér képet tartalmaz. Minden sor bal oldalán és minden oszlop fölött egy számsor áll, amely sorrendben megadja, hogy az adott sorban vagy oszlopban milyen hosszú, összefüggő fekete szakaszok találhatók. A szakaszok között legalább egy fehér mező van. A játékos célja, hogy a számsorok alapján megtalálja a kép összes fekete mezőjét. A játékos egy mezőt kitölthet (feketének jelöl), vagy kizárhat (fehérnek jelöl, X- szel). Egérrel a bal gomb kitölt, a jobb gomb kizár. Húzással egy sorban vagy oszlopban több mező is jelölhető egyszerre. A kitöltést a program azonnal összeveti a képpel. Ha a mező a képen fehér, a kitöltés hibás: a mező nem lesz fekete, hanem automatikusan kizárt lesz, és a játékos egy életet veszít. A játékosnak 3 élete van. A kizárás csak a játékos jelölése, ezt a program nem ellenőrzi. Ha egy sor vagy oszlop összes fekete mezőjét megtalálta a játékos, a program halványítsa el az ahhoz tartozó számsort, és a sor vagy oszlop maradék mezőit jelölje kizártnak. A képeket fájlban tároljuk, a program méretenként legalább 3 képet tartalmazzon. Ezen felül legyen lehetőség adott kitöltési arányú véletlenszerű kép előállítására is.

A program biztosítson lehetőséget új játék kezdésére a táblaméret (5 × 5, 10 × 10, 15 × 15) és a kép megadásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és a tábla nem látható). Ismerje fel, ha vége a játéknak: a játékos győz, ha minden fekete mezőt megtalált, és veszít, ha elfogytak az életei. A program játék közben folyamatosan jelezze ki a játékidőt és az életek számát. Ezen felül legyen lehetőség a játék elmentésére, valamint betöltésére.

Elemzés:

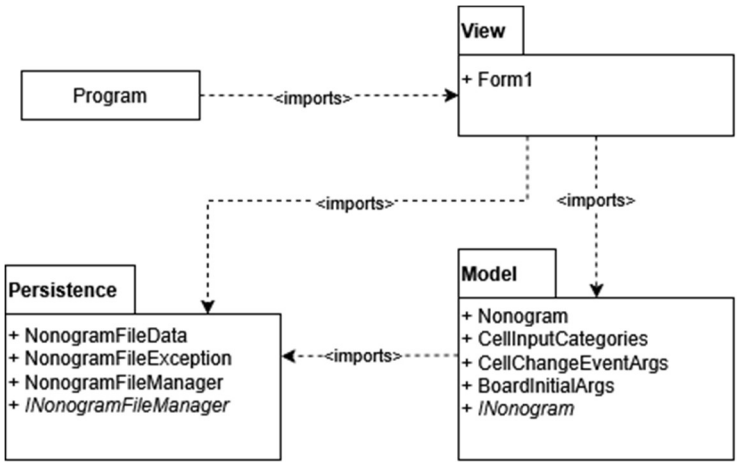
- A játék egy n x n táblából áll, mely lehet 5x5, 10x10 és 15x15 méretű lehet. A program indításakor lehetőség van egy új játék kezdésére vagy egy meglévő játék betöltésére.
- A játék megvalósításához egy egyablakos asztali alkalmazáshoz tervezett Windows Forms felületet használunk.
- Az ablakban elhelyezünk egy menüsort a következő lehetőségekkel:
    - File
        - New
        - Save
        - Load
        - Exit
- Új játék kezdéséhez 3 menüpontra van szükség a *New* kategória alatt: 5x5, 10x10 és 15x15 a három táblamérethez, és ezeken belül pedig 3 + 1 menüpont szükséges az előre elkészített és a véletlenszerűen generált pályák indításához.

- Az alkalmazás felületén szerepel a gombokkal megvalósított tábla, az életek száma és az eddig eltelt idő. A rács jobb oldalán és alján minden sorhoz és oszlophoz tartozik egy szövegdoboz, amik szükségesek a játék megnyeréséhez.
- Egy feulgró dialógusablak jelzi, ha sikerült a rejtvényt megoldanunk *(tehát nyertünk)*, vagy ha életeink elfogytak és emiatt vesztettünk.
- Ebben a játékban az eltelt idő csupán tájékoztató jellegű, nincsen közvetlen befolyása a játék kimenetelére.
- Egy játék mentésénél és betöltésénél szükséges a pálya cellaértékeit, jelenlegi életek értékét és a játszmában eddig eltelt időt elmenteni.
- A felhasználói esetek a következő ábrán láthatóak.



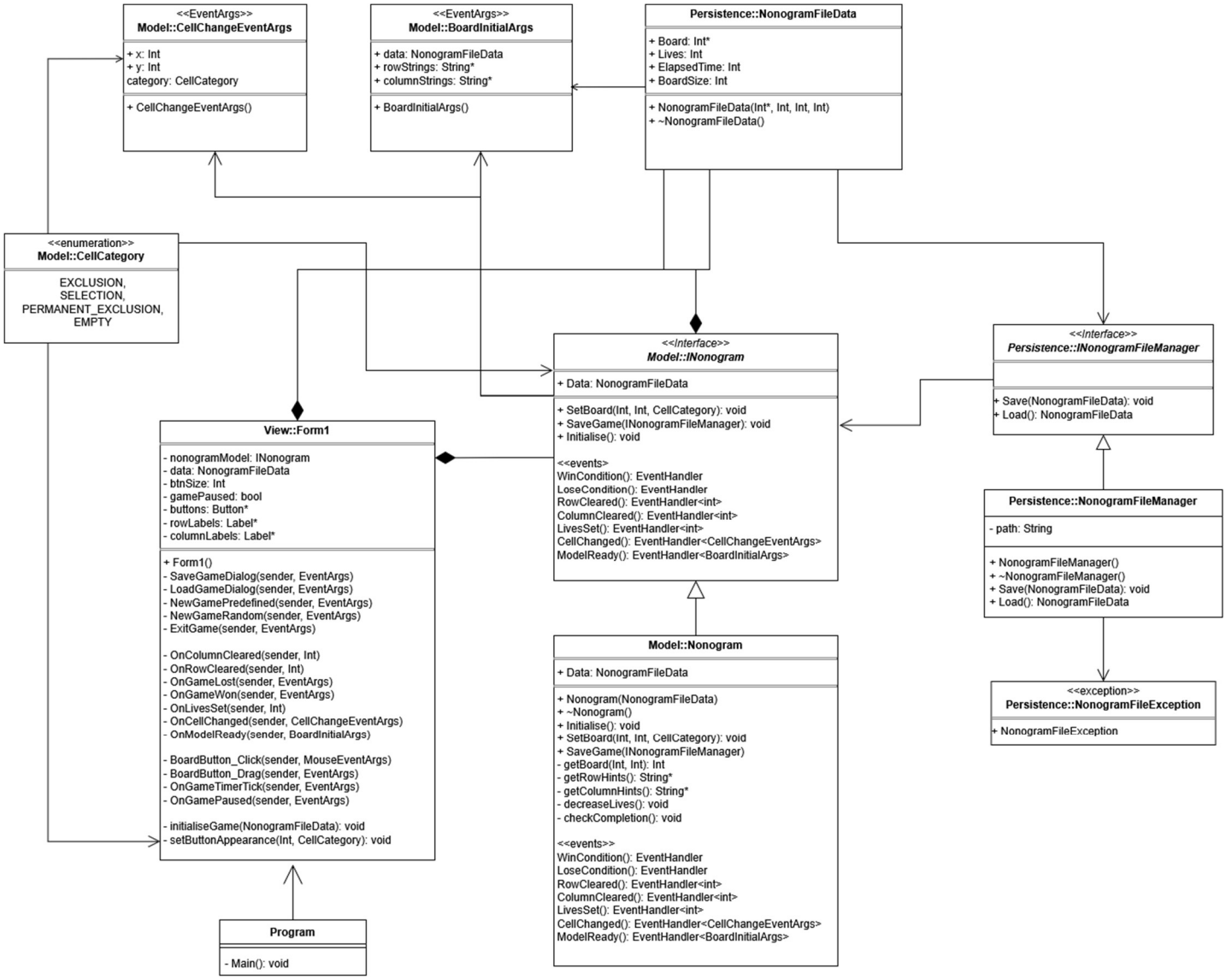
Tervezés

- **Programszerkezet**
  - A program három architekturális rétegből áll. A megjelenítés a **`NonogramView`**, A model a **`NonogramModel.Model`**, és a perzisztencia a **`NonogramModel.Persistence`** névtérben van elhelyezve
  - A program szerkezetileg két projektből áll. A **`Persistence`** és a **`Model`** csomagok egy felülettől független projektben, a **`View`** csomag pedig egy Windows Formsra támaszkodó projektben van elhelyezve.



- **Perzisztencia**
  - A játék pillanatnyi állapota egy **`NonogramFileData`** osztályban él, melyre van hivatkozása a modellnek és a nézetnek is. Ez az osztály tárolja az életek számát, az eltelt időt, a tábla méretét, és magát a táblát.
  - A tábla egy egydimenziós lista mely 4 bites számokat tárol. Egy szám a listában egy cellának az állapotát jelöli a következő módon:
    - 1. bit: a cella része-e a kitalálandó képnek
    - 2. bit: a cellát megjelölte a játékos
    - 3. bit: a cellát a játékos kizárta
    - 4. bit: a cellát a program kizárta, például rossz jelölés esetén.
  - A fájlba való mentés a nézetben kezdi el útját, mely példányosít egy **`NonogramFileManager`** osztályt (mely implementálja az előző interfészt) a megfelelő útvonallal, ezután átadja ezt a modelnek mely meghívja az ebben az osztályban található **`Save(NonogramFileData data)`** metódust.
  - Egy fájl betöltésénél is hasonló dolog történik, csak ott a nézet egy új modelt hoz létre melynek konstruktorába a betöltéskor létrehozott **`NonogramFileData`** objektum kerül.
  - A fájlkezelő osztály egy .ngm kiterjesztésű szöveges fájllal működik. Ennek a fájlnak az első sora az életek száma, a második az eltelt idő másodpercekben, harmadik pedig a tábla celláinak értéke szóközzel elválasztva.
  - Beolvasási hiba esetén a **`NonogramFileException`** kivétel lesz kiváltva.

- **Modell**
  - A modell funkcionális részét a **`Nonogram`** osztály valósítja meg. Az osztály műveletei a benne tárolt **`NonogramFileData`** példányra vannak kihatással.
  - A modell példányosítása a nézeten belül történik meg az **`_initialiseGame`** metódus által, ami abban az esetben, ha a modellt tároló változó nem null, meghívja annak destruktorát a memória szivárgás elkerülésének érdekében. Mivel viszont a modell a **`ModelReady`** eseményen keresztül adja át a nézetnek a kiinduló állapotot *(ami ugye nem lehet még bekötve példányosítás előtt)*, a modell példányosítása után a nézetnek még meg kell hívnia a modell **`Initialise()`** metódusát.
  - Egy gomb megnyomása után a nézet frissítése érdekében a modell meghívja a **`CellChanged`** eseményt **`CellChangeEventArgs`** argumentummal, mely tartalmazza a megváltozott cella **`(x, y)`** pozícióját és a cella új állapotát, melyeket egyszerűség kedvéért a **`CellCategory`** enum absztraktál. Ennek értéke lehet **`EXCLUSION`** *(ha a játékos ki akar zárni egy cellát)*, **`SELECTION`** *(ha a játékos sikeresen megjelölt egy cellát)* és **`PERMANENT_EXCLUSION`** *(ha a játékos helytelenül jelölt meg egy cellát)*. A különbség a kettő kizárási lehetőség között, hogy **`EXCLUSION`** után a játékos még meg tudja jelölni azt a cellát (ha esetleg helytelenül zárta ki), **`PERMANENT_EXCLUSION`** után pedig a játékos már nem tud azzal a cellával interaktálni, hiszen az biztosan kizárt lesz egy helytelen jelölés után.
  - A **`SetBoard`** metódus helyes jelölés után végigfut a táblán és ellenőrzi mely sorok és oszlopok lettek teljesen kitöltve a **`_checkCompletion`** privát metódussal. Ha egy sor vagy oszlop teljesen kitöltött, a **`RowCleared`** vagy a **`ColumnCleared`** esemény ki lesz váltva a kitöltött sor/oszlop indexével. Ha az egész tábla helyesen van kitöltve akkor ez a metódus kiváltja a `WinCondition` eseményt is.
  - Helytelen jelölés esetén a **`_decreaseLives`** metódus fut le, mely kiváltja a **`LivesSet`** eseményt a nézet frissítéséért és ha az életek száma 0 kiváltja a **`LoseCondition`** eseményt.
- **Nézet**
  - A nézetet a **`Form1`** osztály biztosítja, mely tartalmazza a modell egy példányát (**`NonogramModel.Model).`**
  - Az új játék **`toolStripItem`**-eknél az elem **`Tag`** mezője tartalmazza a hozzá tartozó kezdőfájl nevét, a véletlenszerű elemeknél pedig a legenerálandó pálya méretét. Ez a kettő eseménnyel hívja meg a **`NewGamePredefined`** és a **`NewGameRandom`** metódusokat.
  - A pálya dinamikusan létrehozott elemeit a **`_buttons`**, **`_rowLabels`** és **`_columnLabels`** tartalmazza.
  - A gomboknál a bal egérgomb letartásával több cellát is ki lehet jelölni egyszerre, ezen felül egyszerűen meg is lehet őket nyomni.



- **Tesztelés**
  - A modell egységtesztelését a **`NonogramTest`** MSTest projekt valósítja meg.
  - A tesztesetek a modell összes publikus metódusán végigmennek *(kivéve a `Dispose()` metódus, melyet nem szükséges tesztelni).*
  - A modell adatkezelése miatt nagyrészt nem szükséges egy fájlkezelő mockolása, elég egy **`NonogramFileData`** példányt létrehozni. *(Kivéve a mentés teszteléskor)*
  - Az alábbi tesztesetek kerültek megvalósításra:
    - **`ModelInitTest:`** A modell létrehozása után kiváltva lesz-e a kiinduló állapot eseménye.
    - **`SetExclusionTest:`** A játékos által kizárt cellák eseménykiváltása. Egy játékos által kizárt cella egy jobb gombnyomással újra „üres” cellává kell váljon.
    - **`SetCorrectSelectionTest:`** A játékos által helyesen kiválasztott cella eseménykiváltása.

- **`SetIncorrectSelectionTest:`** A játékos által helytelen kiválasztott cella eseménykiváltása. Ilyenkor a **`LivesSet`** esemény is ki kell váltódjon.
- **`SetPermanentExclusionTest:`** A program által véglegesen kizárt cella eseménykiváltása
- **`LoseConditionTest:`** Három helytelen kiválasztás után (mivel a kezdő életek száma 3) a **`LoseCondition`** esemény kell kiváltódjon. Ilyenkor az életek száma is 0 kell legyen.
- **`CompletedRowAutoExclusionTest:`**
- **`CompletedColumnAutoExclusionTest:`** Ha egy sor/oszlop teljesen ki van töltve, akkor a maradék üres cellák véglegesen ki lesznek zárva. Mindegyik cellakizárásról a nézet egy külön eseményt kap.
- **`WinConditionTest:`** Ha minden cellája a képnek meg lett jelölve, akkor a WinCondition esemény ki kell legyen váltva.
- **`SaveTest:`** A modell mentéskor meg kell hívja a fájlkezelő mentés metódusát, ezután pedig már nincs szükség a fájlkezelőre tehát Dispose-olni lehet.
- **`LoadingErrorTest:`** A modell konstruktora **`ArgumentOutOfRangeException`**-t kell dobjon, ha a táblaméret változó nem egyezik meg a tábla lista méretével.